

PROJECT OVERVIEW

GovPay Desk MVP Guide

What the system does, how the workflow runs, and how to work on it.

System	Audience	Revised
GovPay Desk	Developers and maintainers	8 August 2026

CONTENTS

- | | |
|----------------------------------|-------------------|
| 1. Getting started | 6. Project layout |
| 2. The payment file lifecycle | 7. UI conventions |
| 3. Roles | 8. Known gaps |
| 4. The general ledger | 9. Testing |
| 5. Audit trail and notifications | |

1. Getting started

```
composer install
npm install
php artisan migrate:fresh --seed
composer dev          # serves the app, queue and Vite together
```

Open <http://127.0.0.1:8000> and sign in.

Demo accounts

All seeded accounts use the password `password`.

EMAIL	ROLE	WHAT THEY CAN DO
<code>ota@example.com</code>	Administrator	Everything, including approving own vouchers
<code>ama@example.com</code>	Finance Officer	Prepare, submit and pay vouchers
<code>kofi@example.com</code>	Finance Officer	Same as above
<code>vterry@example.net</code>	Approver	Approve or reject others' vouchers

Sign in as **ama@example.com** to prepare a voucher, then **vterry@example.net** to approve it — that is the core loop.

2. The payment file lifecycle

Everything in the system hangs off one chain. A voucher is prepared, approved and paid; a memo is then raised against the paid voucher and filed.

```
VOUCHER                                MEMO
draft → pending → approved → paid → drafted → finalized → printed
      |
      └→ rejected (returned for correction, re-editable)
```

Rules enforced in code, not just the UI:

- Only **draft** and **rejected** vouchers can be edited. An approved or paid voucher is a financial record and is immutable.
- A voucher must be **pending** to be reviewed, and **approved** to be paid.
- **Separation of duties:** whoever prepared a voucher cannot approve it. Only an Administrator may override this.
- A cheque payment requires a cheque number.

These live in `app/Policies/PaymentVoucherPolicy.php` and the form request classes under `app/Http/Requests/PaymentVoucher/`.

3. Roles

Roles are fixed in code (`App\Models\User::roles()`) and enforced by policy. Change someone's role from **Users** → **Edit**.

ROLE	ABILITIES
Administrator	Full access; may approve own vouchers
Approver	Approve/reject submitted vouchers; cannot approve own
Finance Officer	Create, edit, submit drafts; mark approved vouchers paid
Viewer	Read-only

The **Roles & Permissions** page documents this and lists who holds each role.

4. The general ledger

When a voucher is marked **paid**, the system posts a balanced double entry:

```
Dr <expense account, from the voucher's budget line>    amount
Cr <1100 Cash | 1200 Bank, from the payment method>      amount
```

Budget lines are free text, so `app/Services/LedgerPoster.php` maps common phrases to account codes and falls back to **5900 General Expenses**:

BUDGET LINE CONTAINS	POSTS TO
office supplies, stationery	5100 Office Supplies
salaries, payroll	5200 Salaries and Wages
travel, transport	5300 Travel and Transport
utilities	5400 Utilities
maintenance	5500 Repairs and Maintenance
<i>anything else</i>	5900 General Expenses

Cheque and cash payments credit **1100 Cash**; bank transfers credit **1200 Bank**.

Posting is idempotent — a voucher already in the ledger will not post twice. The **Ledger** pages show a running balance check; if debits and credits ever diverge, the page says so in red.

To extend the mapping, edit `BUDGET_LINE_ACCOUNTS` in `LedgerPoster.php`. For a proper system you would replace free-text budget lines with a foreign key to `accounts`.

5. Audit trail and notifications

Every state change writes to `system_logs` via `App\Services\AuditLogger`: who did it, what changed, when, and from which IP. Entries are append-only — nothing in the app updates or deletes them. View them under **System Audit Log**.

`App\Services\Notifier` creates in-app notifications:

- submitting a voucher notifies all Approvers and Administrators
- approving, rejecting or paying notifies whoever prepared it

6. Project layout

```
app/
  Http/Controllers/    one per feature area
  Http/Requests/       validation rules, grouped by model
  Models/              Account, AppNotification, Department, Document,
                      LedgerEntry, Memo, PaymentVoucher, SystemLog, User
  Policies/            PaymentVoucherPolicy – the authorization rules
  Services/            LedgerPoster, AuditLogger, Notifier
resources/js/
  pages/               one folder per feature, matching the route names
  components/ui/       shadcn primitives
  routes/              Wayfinder-generated typed route helpers
tests/Feature/         LedgerPosting, VoucherPolicy, VoucherWorkflow, PagesLoad
```

After changing routes, regenerate the typed helpers:

```
php artisan wayfinder:generate --with-form
```

The `--with-form` flag matters — without it the `.form` variants that auth pages rely on disappear.

7. UI conventions

The interface follows a few rules consistently. Match them when adding pages.

- **Say a thing once.** Page title, card title and section heading should not repeat each other. A status badge on a page already filtered to that status adds nothing.
- **Card padding is set once**, on `<Card className="py-5">`. Do not add `pt-6` to `CardContent` — shadcn's `Card` already has `py-6`, and the two stack into a large dead band.
- **Spacing scale** on forms: `space-y-5` between sections, `space-y-2` from a heading to its fields, `gap-y-3.5` between field rows, `space-y-1.5` from a label to its input.
- **Money** is right-aligned, `tabular-nums`, two decimals, with an em-dash for zero. Tables that show money show totals.

- **Every list** needs an empty state that distinguishes "nothing here yet" from "nothing matches your filter", and offers a way out of the filter.
 - **Every paginated list** renders pagination. A count that says "10 of 13" with no way to reach the other 3 is a bug.
 - **Errors** appear both inline and in a summary at the top of long forms, with each summary line linking to its field.
 - **Dates** use `en-GB` formatting (`3 August 2026`).
 - Colour is never the only signal — pair it with an icon or text.
-

8. Known gaps

Honest list of what is not built:

- **Documents are read-only.** The table and model exist, but there is no upload. Files only appear via the seeder.
 - **No voucher detail page.** There is no `payment-vouchers.show` route; lists link to Edit.
 - **Memos have no edit or print output.** They can be created, finalized and marked printed, but not edited, and "printed" is a status rather than a generated document.
 - **Ledger is one-directional.** Vouchers post to it; there is no manual journal entry, no reversal, and no period close.
 - **GIFMIS is not implemented** and has been removed from the app entirely.
 - **Search is server-side and unindexed** — `LIKE` queries, fine at this size.
 - **New users get the password** `password` . There is no invite or reset flow for admin-created accounts.
-

9. Testing

```
php artisan test           # 81 tests
npx tsc --noEmit           # typecheck
npx vite build             # production build
./vendor/bin/pint          # PHP formatting
```

The suite covers ledger posting (balance, account mapping, idempotency), the authorization rules including a tampering attempt against a paid voucher, an end-to-end draft→paid workflow, and a smoke test that every page returns 200.